

OFFLINE AUTHORIZING STANDARD

Study Tracker Follow Along Authoring Standard

A standalone specification that can be uploaded to ChatGPT, Gemini or another capable model to create complete beginner-friendly Follow Along content without access to the Study Tracker repository.

IMPORTANT

The external model creates an educational manuscript. It must not claim that it created an app-ready handoff fingerprint, Author draft, release candidate or publication. Those remain controlled local-app operations.

Prepared for Shaun Estcourt | Version 1.0 | 16 August 2026

1. How to use this PDF

1. Open a new ChatGPT or Gemini conversation.
2. Upload this PDF as the governing specification.
3. Send the short master request on page 17 with your chosen topic and learning outcome.
4. Ask the model to return the two offline deliverables described in section 14.
5. Review the readable preview. Correct educational content before involving Study Tracker.
6. Give the approved manuscript to an app-aware local Codex task for schema conversion, deterministic validation and controlled import.

What this saves

- No OpenAI API key is required when the drafting model is used through a normal ChatGPT or Gemini plan.
- The expensive content-writing stage happens outside the Author Assistant API script.
- Only the much smaller deterministic conversion and validation stage needs repository access.
- The same specification can be reused for Terraform, AWS, networking, Kubernetes and future exam workspaces.

ONE DOCUMENT

Upload this entire PDF. Do not copy only isolated pages, because safety, cleanup and output rules depend on each other.

What the external model cannot do

- It cannot inspect the current app schema, database state, Shared Drafts or candidate queue.
- It cannot generate a trustworthy Study Tracker fingerprint without the local fingerprint helper.
- It cannot bind an Author identity, import, approve or publish.
- It must not invent confirmation that local validation succeeded.

2. Required input from the learner

Before writing, the external model must have the following facts. If a fact is missing, it may ask a short question rather than inventing it.

Input	Example
Programme title	Terraform Import and Maintenance Follow Along
Topic or service	Terraform import blocks with AWS Systems Manager Parameter Store
Exam workspace	HashiCorp Terraform Associate 004
Learner level	Beginner to Intermediate
Outcome	Import, inspect, repair drift and safely destroy one existing object
Region or scope	eu-west-2 or global
Routes	Graphical/Console and CLI/Terraform
Resource prefix	fa-terraform-import-maintenance
Constraints	No existing infrastructure; training account only

Default assumptions

- The learner starts with no infrastructure created for this lab.
- The learner may not know how to open a terminal, create a folder, create a file or save it with the correct extension.
- The learner uses a disposable training account and must not use the root account for routine work.
- Every resource has a harmless fixed training name wherever the provider permits one.
- Durations are optional. Use Self-paced when no genuine estimate exists.

3. Non-negotiable teaching standard

- Write for a learner performing the task for the first time.
- Explain what the learner is doing, why it matters and how to recognise success.
- Use one concrete action per checkbox. Do not compress several clicks into vague prose.
- Name the exact page, menu, button, field, value and Region whenever relevant.
- Supply every required policy, Terraform file, YAML document, JSON body, shell script and command in full.
- Explain every placeholder before first use. Prefer fixed harmless names over placeholders.
- Include a visible expected result after each meaningful step.
- Include complete verification and complete reverse-dependency cleanup.
- Do not use placeholder, catalogue, future work, add later, supplied elsewhere or deferred wording.
- Do not assume existing VPCs, subnets, roles, profiles, repositories, folders, files, state or credentials unless the user explicitly says they exist.

QUALITY TEST

A learner should be able to complete the lab using only the generated manuscript and its linked official documentation.

Forbidden shortcuts

- Configure the service as required.
- Create the necessary permissions.
- Use your existing network.
- Paste the supplied JSON when no JSON block is included.
- Run the usual Terraform commands.
- Delete the resources when finished.

4. Programme structure

Organise the programme in a logical learning journey. Four phases is a useful minimum for a substantial Follow Along, but use as many phases and tasks as the subject genuinely requires.

Recommended phase pattern

1. Prepare - explain scope, tools, identity, Region, names and safety boundaries.
2. Create - build the smallest complete working environment from nothing.
3. Understand - inspect files, resources, state, outputs and important decisions.
4. Change and troubleshoot - make a controlled change, observe behaviour and repair a realistic fault.
5. Verify - prove the intended result using visible evidence.
6. Cleanup - delete every lab-created item in reverse dependency order and verify absence.

Every task requires

- Stable unique task ID and phase ID.
- Title, feature, goal, why it matters and Easy, Medium or Hard difficulty.
- Prerequisites referencing earlier task IDs only.
- Explicit mode availability for graphical/Console and CLI/Terraform routes.
- Complete steps, expected results, warnings where needed and at least one verification check.
- Official source IDs that resolve to the source list.

5. Console and graphical route

A graphical route is not a summary of the CLI route. It must be independently usable.

1. State where to sign in and how to confirm the intended training account.
2. State how to select the required Region.
3. Name the service to enter in the Console search box.
4. Name each navigation item and button in click order.
5. Give the exact value for every field and explain important choices.
6. Explain safe defaults and settings that must remain unchanged.
7. Show where generated IDs, status values, logs, tags or outputs appear.
8. Include a visible success condition before continuing.
9. Explain the Console cleanup path and exact confirmation text.

Good instruction

EXAMPLE

In the upper-right Region selector choose Europe (London) eu-west-2. In the Console search box enter Systems Manager, open it, choose Parameter Store in the left navigation, then choose Create parameter.

Weak instruction

DO NOT USE

Open SSM and create the parameter with the correct settings.

6. CLI and Terraform route

- Show how to open the correct terminal for Windows PowerShell and Bash-based environments where both are supported.
- Show how to create and enter the exact lab folder, then verify the current location.
- Show how to create every file. Include the complete content directly after the instruction.
- Use one executable command per command entry. Do not chain commands with semicolons, && or pipes unless the pipe itself is the concept being taught.
- Explain what each command does, why it is run and the expected output.
- For Terraform, teach init, fmt, validate, plan, saved-plan review, apply and verification in order.
- Never apply an unreviewed plan. Use a named saved plan for consequential changes and teardown.
- Explain configuration, provider cache, lock file, state, plan files and outputs when they first appear.
- Provide separate PowerShell and Bash commands when syntax differs materially.

Terraform file rule

Never say Save the supplied file unless the complete file is present in the same task. Terraform and HCL must use a text block, not a JSON block.

```
EDITABLE BLOCK: main.tf
language: text

terraform {
  required_version = ">= 1.5.0"
}

# Complete configuration continues here.
END EDITABLE BLOCK
```

7. Editable code and policy blocks

Required block fields

- A clear title or filename.
- A correct language value.
- Complete copyable content with readable line breaks and indentation.
- An instruction explaining exactly where the learner pastes or saves it.
- Any safe values that must be replaced, explained before the block.

Language rules

- Use language json only when the block is valid JSON.
- Use language text for Terraform HCL, YAML, PowerShell, Bash, user data and other non-JSON content.
- Do not convert multiline code into one long line.
- Do not hide required content in prose or an external source link.

IAM POLICY

If a lab creates a customer-managed IAM policy, provide the complete valid policy JSON in a visible editable block. Explain its scope and why each permission group exists.

8. Identity, credentials and secrets

- Do not use the AWS root user for routine lab actions.
- Prefer IAM Identity Center and temporary credentials when the learning goal permits it.
- If the lab specifically teaches a temporary IAM user, an administrator creates the narrowly scoped user and policy, and cleanup deletes them.
- Never put a password, secret access key, session token, private key or other credential in the manuscript, source control, screenshots, chat or Study Tracker.
- Never ask the learner to paste a credential into a Saved Resource Capture field.
- Access-key values may be entered only into the protected AWS CLI configure prompts, then the creation screen must be closed.
- Do not invent a least-privilege policy. Derive permissions from the exact lab actions and official documentation, then explain any unavoidable Resource star.

SECRET RULE

The output must contain zero real credentials. If the user supplies a credential, do not repeat it; tell the user to revoke or rotate it.

Saved resource capture

Only recommend saved bindings for genuinely variable, non-secret values that later instructions interpolate, such as an automatically generated instance ID, ARN or mapping UUID. Do not capture fixed names, local folders, profile names, policy names, user names or access keys.

9. Resource naming and lifecycle

- Use one unique fixed prefix across the programme.
- List every created cloud, identity, networking and local resource.
- Mark whether a name is fixed or generated.
- Mark whether a resource can incur charges.
- Record dependencies so cleanup can be reversed safely.
- Do not tell the learner to select or delete an unnamed resource.
- When a value is generated, explain exactly where to find it and use a safe symbolic reference in later prose.

Portable resource record

```
{
  "id": "resource-app-instance",
  "label": "Application EC2 instance",
  "exactName": "fa-example-web-a",
  "generatedValue": "instance ID",
  "captureAllowed": true,
  "sensitive": false,
  "chargeable": true,
  "createdByTaskId": "task-create-instance",
  "dependsOn": ["resource-public-subnet", "resource-security-group"]
}
```

10. Verification and expected results

Verification must prove that the learner achieved the goal. Merely repeating the action is not verification.

- Use visible Console state, command output, HTTP response, log event, metric, plan summary or state listing.
- State the exact expected result and important values.
- Include failure clues and the safest next check when practical.
- Keep Console verification and CLI verification separately identifiable.
- Use at least one verification check per task and as many as the task genuinely needs.

Verification record

```
{
  "id": "verify-imported-address",
  "title": "Verify the imported state binding",
  "route": "cli",
  "instruction": "Run terraform state list from the initialized lab folder.",
  "expectedResult": "Exactly aws_ssm_parameter.application_mode is printed."
}
```

11. Cleanup and cost safety

Cleanup is part of the learning outcome, not a short final note.

1. Inventory every resource created by the programme.
2. Identify dependencies and start with the most dependent workload.
3. Create and inspect a saved destroy plan where Terraform manages the resource.
4. Delete cloud resources before deleting credentials, state or local configuration.
5. Verify cloud absence while the required verification identity still works.
6. Delete temporary access keys, users, policies and local profiles only after cloud verification.
7. Delete local state, plan files and the exact lab folder last.
8. Verify that unrelated resources and profiles remain unchanged.
9. Finish with one explicit programme-cleanup acknowledgement.

Required cleanup properties

```
{  
  "manualOnly": true,  
  "ordering": "reverse_dependency",  
  "completionGate": "acknowledgement"  
}
```

**DELETION
SAFETY**

Every deletion instruction must name the exact target and include a visible verification. Never use a broad wildcard or unresolved path.

12. Official source rules

- Use primary official documentation only unless the user explicitly approves another source.
- For AWS use docs.aws.amazon.com and other official AWS documentation domains.
- For Terraform and HCP Terraform use developer.hashicorp.com.
- For another vendor use that vendor's official documentation.
- Do not rely on search-result snippets. Open and read the supporting page.
- Every source record lists the task IDs it supports.
- Every task lists the source IDs supporting it.
- Do not cite a source for a claim it does not support.

Source record

```
{
  "id": "source-terraform-import-block",
  "title": "Import existing resources",
  "url": "https://developer.hashicorp.com/terraform/language/import",
  "publisher": "HashiCorp",
  "purpose": "Supports declarative import-block behaviour.",
  "taskIds": ["task-write-import", "task-apply-import"]
}
```

13. Portable manuscript format

The external model must return one complete JSON manuscript using this top-level structure. This is deliberately app-independent; a local converter will map it to the current Study Tracker schema.

```
{
  "manuscriptVersion": "1.0",
  "programme": {
    "title": "...",
    "suggestedProgrammeId": "...-learning-path",
    "topic": "...",
    "examWorkspace": "...",
    "learnerLevel": "...",
    "outcome": "...",
    "region": "...",
    "resourcePrefix": "...",
    "selfPaced": true
  },
  "sources": [],
  "phases": [],
  "tasks": [],
  "resources": [],
  "programmeCleanup": [],
  "warnings": {
    "cost": "...",
    "safety": "...",
    "credentials": "...",
    "region": "..."
  },
  "qualityReport": {},
  "boundaries": {
    "appReady": false,
    "fingerprinted": false,
    "imported": false,
    "published": false
  }
}
```

14. Required offline deliverables

Deliverable 1 - complete manuscript

Filename: **offline-follow-along-manuscript.json**

- Valid UTF-8 JSON.
- Complete programme, phases, tasks, steps, commands, editable blocks, verification, sources, resources, warnings and cleanup.
- No comments outside JSON and no truncated sections.

Deliverable 2 - readable review

Filename: **offline-follow-along-preview.md**

- Programme summary and learner outcome.
- Numbered phases and tasks.
- All Console instructions, CLI commands and editable blocks.
- Expected results, verification and cleanup.
- A final quality report listing counts and unresolved issues.

**STOP
BOUNDARY**

After producing these two files, the external model must stop. It must not fabricate a Step 90 package, acceptance audit, fingerprint, Author identity, candidate ID, approval or publication.

15. Self-review before returning files

1. Confirm every task has a unique ID and belongs to exactly one phase.
2. Confirm prerequisites reference existing earlier tasks and contain no cycle.
3. Confirm every supported route is complete and independently usable.
4. Confirm every required file or policy appears in full.
5. Confirm every command has a purpose and visible expected result.
6. Confirm every task has verification.
7. Confirm every source-to-task and task-to-source link is reciprocal.
8. Confirm cleanup names every created item and follows reverse dependency order.
9. Confirm cloud cleanup occurs before credential, state and local-file cleanup.
10. Confirm the output contains no real credential or sensitive saved binding.
11. Confirm there is no placeholder, deferred or future-content wording.
12. Confirm the model reports any genuine uncertainty instead of inventing details.

Required quality report

- Phase count, task count and checkbox count.
- CLI command count, editable-block count and verification count.
- Cleanup-item count and official-source count.
- Missing or uncertain items, which must be zero for a complete manuscript.

16. Local Study Tracker handoff

When the manuscript is approved educationally, give both files to a local Codex task that can access the Study Tracker repository.

- 1. Read the current verified builder and schema from the local repository.
- 2. Convert the portable manuscript without weakening or summarising its content.
- 3. Create new stable IDs and a unique programme ID.
- 4. Run the current planning, content and review validators.
- 5. Calculate the fingerprint using the app's deterministic helper.
- 6. Show the exact fingerprint and stop for human acceptance.
- 7. Create the acceptance audit only after exact fingerprint approval.
- 8. Use the existing controlled Author, Shared Draft, candidate, Approver and publishing workflow.

LOCAL AUTHORITY	Only the app-aware local stage may claim that the package is import-ready. Only the protected database workflow may approve or publish it.
----------------------------	--

17. Short master request

Upload this PDF, then send the following request. Replace the bracketed values only.

```
Read the complete attached Study Tracker Offline Follow Along Authoring Standard and treat it as the governing contract. Create a complete Follow Along manuscript for [TOPIC OR SERVICE], aimed at [LEARNER LEVEL] learners studying [EXAM OR WORKSPACE]. The learner starts with no lab infrastructure. The required outcome is: [OUTCOME]. Use [REGION OR GLOBAL SCOPE] and the fixed training prefix [RESOURCE PREFIX]. Include complete graphical or Console guidance where relevant, complete CLI or Terraform guidance, every required editable file or policy, visible expected results, verification, official primary sources and reverse-dependency cleanup. Return exactly offline-follow-along-manuscript.json and offline-follow-along-preview.md. Do not claim app validation, fingerprinting, import, approval or publication.
```

Example topic request

```
Topic: HCP Terraform  
Learner level: Beginner  
Exam workspace: HashiCorp Terraform Associate 004  
Outcome: Create an HCP Terraform organisation, project and VCS-backed workspace; configure non-secret and sensitive variables; run, review and apply a remote plan; inspect remote state and locking; diagnose one failed run; then safely destroy the training infrastructure and remove the workspace.  
Scope: AWS eu-west-2 plus global HCP Terraform resources  
Resource prefix: fa-hcp-terraform
```

18. Final acceptance checklist

- The manuscript teaches rather than merely instructs.
- The learner creates required infrastructure from nothing.
- Every click, field, value, file and command is available.
- Console and CLI routes are both complete when relevant.
- All code blocks preserve readable formatting.
- No secrets are requested, displayed or stored.
- Only genuinely variable non-secret IDs are eligible for saved capture.
- Every action has an expected result or verification path.
- Every created item appears in cleanup.
- Cleanup order is safe and visibly verified.
- Only official documentation supports technical claims.
- The external model stops after the two offline files.

READY

If every item above is true, the educational manuscript is ready for local Study Tracker conversion and deterministic validation.

End of standard.